Operating Systems

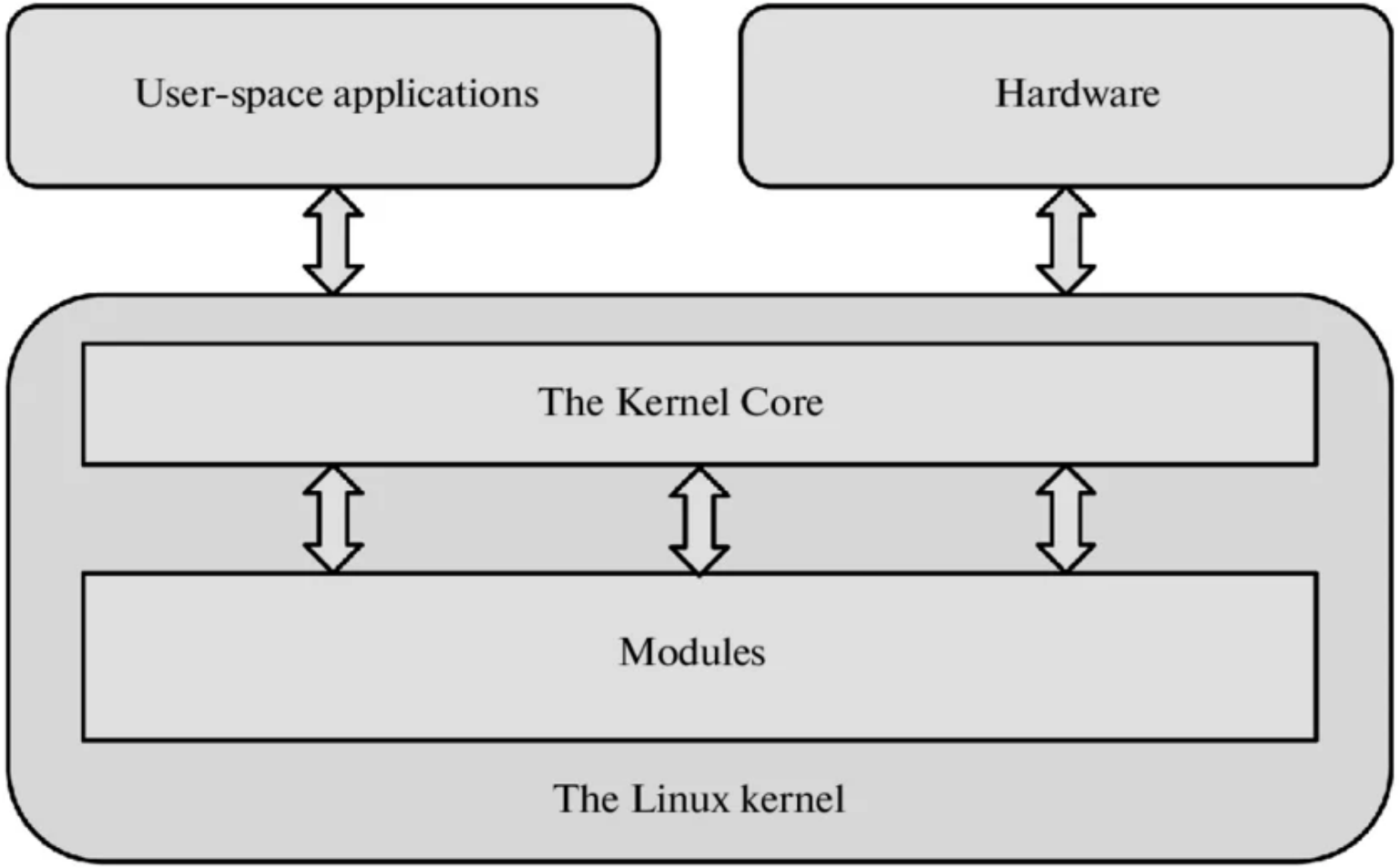
Dmitry Zaitsev

Lecture 18: Design of Linux kernel loadable modules.

What is Linux Kernel Loadable Module?

A Linux Kernel Loadable Module (LKM) is a piece of code that can be loaded into the Linux kernel at runtime without recompiling the whole kernel

LKM within kernel



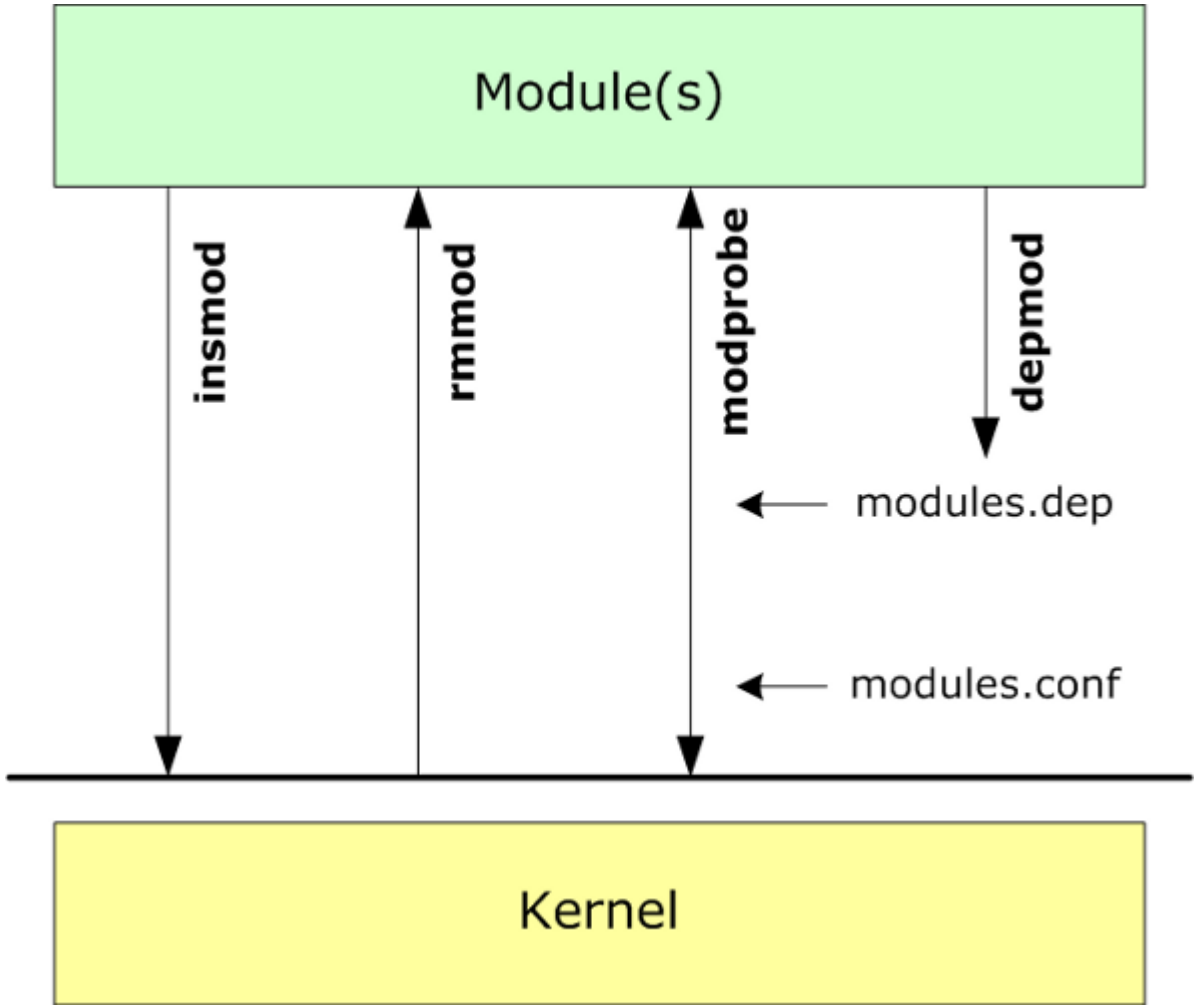
Linux: Loadable Kernel Module (LKM)

- an object file containing code that extends the functionality of the Linux kernel at runtime
- LKMs can be loaded and unloaded without recompiling the kernel or rebooting the system
- Dynamic Loading: LKMs are typically stored as .ko (kernel object) files in /lib/modules/ and are inserted into the running kernel when needed
- Kernel Space Execution: Once loaded, a module becomes an integral part of the kernel and runs in Ring 0 (privileged mode), meaning it has full access to the system

Common Uses of KLM

- Device Drivers: Support for hardware like graphics cards, Wi-Fi adapters, and printers.
- Filesystem Drivers: Support for specific formats such as ext4, NTFS, or NFS.
- Network Protocols: Implementing specific protocols like TCP/IP, e6 or specialized VPN features.
- System Calls: Adding or overriding the standard interface between user applications and the kernel.

Basic commands to maintain LKM



Essential Commands for KLM

- lsmod: Lists all currently loaded modules
- modprobe: The preferred way to load or unload modules because it automatically handles dependencies (loading other required modules first)
- insmod: Manually inserts a specific .ko file into the kernel
- rmmod: Removes a module from the running kernel

LKM vs. Built-in

Feature Loadable	Module (LKM)	Built-in
Boot Requirement	Loaded after the root filesystem is mounted	Available immediately at boot
Memory	Uses memory only when loaded	Permanently occupies memory
Flexibility	Updated/replaced without a reboot	Requires a full kernel recompile and reboot to update
Performance	Minor penalty due to potential memory fragmentation	Slightly faster as it is in contiguous memory

Core Principles of LKM Operation

- **Dynamic Loading/Unloading:** Unlike monolithic kernels that require recompilation to add features, LKMs are loaded on demand (using insmod or modprobe) and unloaded (using rmmod) to manage system resources efficiently.
- **Ring 0 Operation:** LKM code runs in the kernel's privileged mode (ring 0).
- **File Extension:** Compiled kernel modules use the .ko extension.
- **Structure:** Modules typically require an *init* function (run upon insertion) and an *exit* function (run upon removal).

A new system call implementation

- Find next available number in system calls table
- Specify parameters of new system call
- Init function should register a new system call
- Exit function should unregister a new system call
- Develop a user side wrapper function with convenient list of parameters
- The module should copy parameters between user and kernel space

Alternative ways of KLM communication

- User defined pseudo devices
- Dedicated directories of kernel pseudo filesystems /proc and /sys
- Special pseudo filesystems debugFS, RelayFS, and TraceFS
- Sockets
- Common segments of memory
- Signals to user processes

Communication with user processes

- Character Device Interface (/dev) – the most traditional and common method. Interaction via file operations
- ioctl() interface for sending commands or structured data
- /proc filesystem. Modules can create entries under: /proc/my_module
- sysfs (/sys). Modern Linux prefers sysfs for configuration interfaces under: /sys/module/my_module/
- Netlink Sockets. Used for asynchronous communication between kernel and userspace
- mmap() Shared Memory. User program maps kernel buffers
- Signals to User Processes. Kernel can notify user processes using signals

Character Device Interface

- The kernel module registers a **character device driver**, which creates a device file like:

/dev/mydevice

- User programs interact with it using standard system calls
- Register device: register_chrdev(major, "mydevice", &fops);
- Userside:

```
int fd = open("/dev/mydevice", O_RDWR);
write(fd, "hello", 5);
read(fd, buffer, sizeof(buffer));
ioctl(fd, CMD, arg);
```

ioctl() Interface

- Kernel module code:

```
static long device_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {
    switch(cmd) {
      case MY_IOCTL_CMD:
          copy_from_user(&data, (void __user *)arg, sizeof(data));
          break;
    }
    return 0;
}
```

- User code:

```
ioctl(fd, MY_IOCTL_CMD, &data);
```

/proc Filesystem

- Kernel module code creates entry under /proc/my_module:

```
proc_create("my_module", 0666, NULL, &proc_fops);
```

- User accesses the entry to read and write with:

```
>cat /proc/my_module
>echo "data" > /proc/my_module
```

Netlink Sockets

- Kernel module code:

```
struct sock *nl_sk = netlink_kernel_create(&init_net,
                                           NETLINK_USERSOCK,
                                           &cfg);
```

- Send message:

```
nlmsg_unicast(nl_sk, skb, pid);
```

- User code:

```
socket(AF_NETLINK, SOCK_RAW, NETLINK_USERSOCK);
```

mmap() Shared Memory

- Both kernel and user process access the same memory pages.
- Kernel module code:

```
static int device_mmap(struct file *filp,
                       struct vm_area_struct *vma)
{
    remap_pfn_range(...);
}
```

kbuild

- the build system used by the Linux kernel. Modules must use kbuild to stay compatible with changes in the build infrastructure and to pick up the right flags to the compiler
- ***https://docs.kernel.org/kbuild/modules.html***
- To build external modules, you must have a prebuilt kernel available that contains the configuration and header files used in the build
- The kernel must have been built with modules enabled
- An alternative – prepare a kernel source tree for building external modules

```
>make modules_prepare
```

Basic commands for building KLM

Build an external module

>make -C <path_to_kernel_dir> M=$PWD

Build against the running kernel

>make -C /lib/modules/`uname -r`/build M=$PWD

To install the module(s) just built, add the target “modules_install” to the command

>make -C /lib/modules/`uname -r`/build M=$PWD modules_install

Linux kernel loadable module design example

- Converts case of the input string. Character device interface /dev/zdev
- Be sure the current kernel table of symbols is accessible
- Create a dedicated directory for the kernel loadable module design and use it

```
>mkdir kmod
>cd kmod
>nano zdev.c
>nano Makefile
```

Create kernel loadable module (1)

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "zdev"
#define BUF_SIZE 128

static char kernel_buf[BUF_SIZE];
static int major;

// Read from module
static ssize_t my_read(struct file *file, char __user *buf, size_t len, loff_t *offset)
{
    ssize_t ret;
    ret = simple_read_from_buffer(buf, len, offset, kernel_buf, strlen(kernel_buf));
    return ret;
}
```

Create kernel loadable module (2)

```
// Write to module
static ssize_t my_write(struct file *file, const char __user *buf, size_t len,
                        loff_t *offset)
{
    ssize_t to_copy = min(len, (size_t)(BUF_SIZE - 1));
    ssize_t not_copied;

    not_copied = copy_from_user(kernel_buf, buf, to_copy);
    if (not_copied > 0)
        return -EFAULT; // Failed to copy some bytes

    kernel_buf[to_copy] = '\0'; // Null terminate

    // process data
    for(int i=0; i<to_copy; i++) kernel_buf[i]=convertCase(kernel_buf[i]);
    return to_copy;
}
```

Create kernel loadable module (3)

```
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .read = my_read,
    .write = my_write,
};


static int __init zdev_init(void)
{
    major = register_chrdev(0, DEVICE_NAME, &fops);
    if (major < 0) {
        printk(KERN_ERR "zdev: failed to register device\n");
        return major;
    }

    printk(KERN_INFO "zdev: module loaded with major %d\n", major);
    return 0;
}
```

Create kernel loadable module (4)

```
static void __exit zdev_exit(void)
{
    unregister_chrdev(major, DEVICE_NAME);
    printk(KERN_INFO "zdev: module unloaded\n");
}

module_init(zdev_init);
module_exit(zdev_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Dmitry Zaitsev");
MODULE_DESCRIPTION("New character device zdev");

// Load module to kernel
// sudo insmod zdev.ko
// Interact with module
// sudo mknod /dev/zdev c <major_number> 0
// echo "Hello Kernel" | sudo tee /dev/zdev
// cat /dev/zdev
```

Create Makefile to build kernel loadable module

```
obj-m += zdev.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
	make -C $(KDIR) M=$(PWD) modules

clean:
	make -C $(KDIR) M=$(PWD) clean
```

Build, install, and test LKM

```
% build module
>make
% load module into kernel
>sudo insmod zdev.ko
% find inode number
>dmesg | tail
[56008.310446] zdev: loading out-of-tree module taints kernel.
[56008.310893] zdev: module loaded with major 510
% create zdev with inode number
>sudo mknod /dev/zdev c 510 0
% write to zdev
>echo "Hello Kernel" | tee /dev/zdev
Hello Kernel
% read from zdev
>cat /dev/zdev
hELLO kERNEL
```

Create an application to test module

```
#include <stdio.h>
#include <fcntl.h>    // open()
#include <unistd.h>   // read(), write(), close()
#include <string.h>   // strlen()
#include <errno.h>    // errno, perror()

int main(){
  int fd = open("/dev/zdev", O_RDWR);
  char buf[128];

  for(;;){
    printf("input string>");
    fgets(buf,127,stdin);
    write(fd, buf, strlen(buf));
    read(fd, buf, sizeof(buf));
    buf[127]='\0';
    printf("Kernel said: %s\n", buf);
  }
  close(fd);
}
```

Test module, using the application

```
dima@hare:~/kmod$ ./usezdev
input string>Dima Zaitsev
Kernel said: dIMA zAITSEV

input string>
```